

Числа с плавающей точкой

Повторение: представление дробных чисел

- Как в компьютере представлены целые числа?
- Какие проблемы будут, если также представить дробные числа?

Повторение: представление дробных чисел

- Как в компьютере представлены целые числа?
- Какие проблемы будут, если также представить дробные числа?
- Какие решения этих проблем вы знаете?

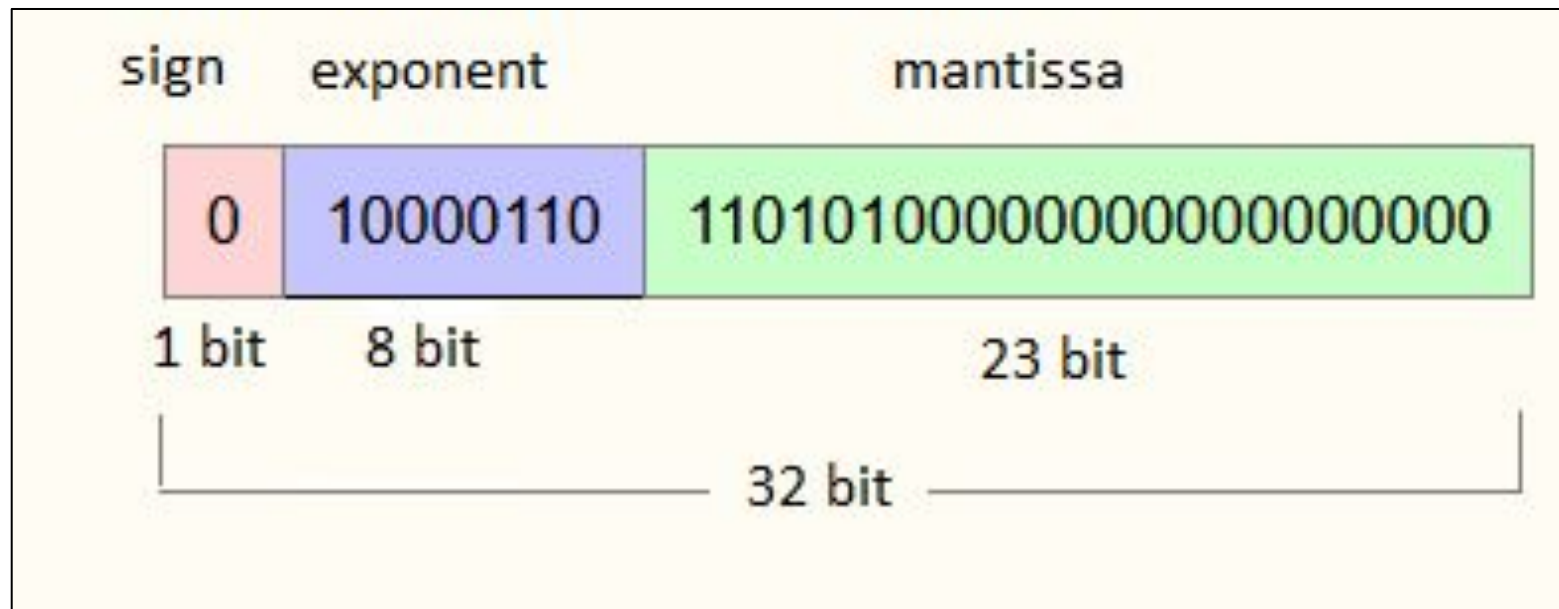
Стандарт IEEE754

Если на каждом процессоре использовать свою систему дробных чисел, то:

- программы станут непереносимы
- программы будут давать разные результаты (разные ошибки округления)

Поэтому был введен единый стандарт представления чисел с плавающей точкой – IEEE754

Стандарт IEEE754



Стандарт IEEE754

- Порядок хранится с учетом смещения. Для 32х битных fp это $2^8 - 1 = 127$. Это позволяет хранить только положительные значения экспоненты. Иначе экспоненту пришлось бы хранить как знаковое число.
 - $E = e - 127$
- Большинство чисел хранится в **нормализованной форме**:
 - $M = 1.xxxx$
- Когда невыгодно хранить числа в нормализованной форме и почему?

Стандарт IEEE754

- Когда невыгодно хранить числа в нормализованной форме и почему?
- Когда число близко к 0, нормальная форма плохо подходит. Тогда используются **денормализованные числа**:
 - $M = 0.xxxx$

Особые значения

Существует 3 особых значения. Все они выделяются тем, что $e = 111\dots111$

- **+inf**: $s = 0, m = 0\dots0$
- **-inf**: $s = 1, m = 0\dots0$
- **NaN**: $m \neq 0\dots0$

```
1 float pos_inf = 1.0f / 0.0f;    // +inf
2 float neg_inf = -1.0f / 0.0f;   // -inf
3 float nan_val = 0.0f / 0.0f;    // NaN
```


Тренировка для понимания (я устал)

Пусть есть 8-битное число с плавающей точкой:

- Экспонента занимает 3 бита
- Мантисса 4 бита

Переведите следующие числа в двоичное представление согласно стандарту IEEE754

<i>s</i>	<i>e</i>	<i>m</i>
$1\frac{1}{4}$		
$-\frac{3}{8}$		
$-\frac{2}{3}$		
$-\infty$		

Тренировка для понимания (я устал)

Пусть есть 8-битное число с плавающей точкой:

- Экспонента занимает 3 бита
- Мантисса 4 бита

Переведите следующие числа в двоичное представление согласно стандарту IEEE754

	<i>s</i>	<i>e</i>	<i>m</i>
$1\frac{1}{4}$	0	011	0100
$-\frac{3}{8}$	1	001	1000
$-\frac{2}{3}$	1	010	0101
$-\infty$	1	111	0000

Математические операции

IEEE754 стандартизирует не только как числа хранятся, но и как с ними работать. Общий процесс выглядит так:

1. Распаковка числа: извлекаются S, E, M
2. Проверка специальных значений: NaN, +/- inf
3. Выполнение арифметики (рассмотрим далее)
4. Нормализация: результат приводится к $1.xxxx * 2^E$
5. Округление
6. Упаковка: S, E, M собираются обратно

Математические операции: умножение

1. Мантиссы перемножаются
2. Экспоненты складываются
3. Над знаковыми битами работает XOR

$$A = 5 / 4 = 0_011_0100$$

$$B = -3 / 8 = 1_001_1000$$

$$A * B = ?$$

Математические операции: деление

1. Мантиссы делятся
2. Экспоненты вычитаются
3. Над знаковыми битами работает XOR

$$A = 5 / 4 = 0_011_0100$$

$$B = -3 / 8 = 1_001_1000$$

$$A / B = ?$$

Математические операции: сложение

1. Меньшее по модулю число сдвигается так, чтоб экспоненты выровнялись: $E1 == E2$
2. Мантиссы складываются
3. Знаковый бит берется исходя из результатов

$$A = 5 / 4 = 0_011_0100$$

$$B = 3 / 8 = 0_001_1000$$

$$A + B = ?$$

Математические операции: вычитание

1. Меньшее по модулю число сдвигается так, чтоб экспоненты выровнялись: $E1 == E2$
2. Мантиссы вычитаются
3. Знаковый бит берется исходя из результатов

$$A = 5 / 4 = 0_011_0100$$

$$B = - 3 / 8 = 1_001_1000$$

$$A - B = ?$$

Что должен знать каждый программист

1. $\text{NaN} \neq \text{NaN}$
2. $+0$ и -0 это разные fp -числа
3. Деление на 0 для fp обычно не завершает программу, а лишь выставляет флаги
4. fp нельзя сравнивать напрямую, только через eps
5. Перед поиском суммы в массиве fp , его лучше отсортировать
6. Сложение fp неассоциативно: $a + (b + c) \neq (a + b) + c$